

# FlashInfer for LLM Serving

Zihao Ye<sup>\*12</sup> Lequn Chen<sup>3</sup> Ruihang Lai<sup>4</sup> Wuwei Lin<sup>2</sup> Yineng Zhang<sup>5</sup> Stephanie Wang<sup>1</sup>  
Tianqi Chen<sup>24</sup> Baris Kasikci<sup>1</sup> Vinod Grover<sup>2</sup> Arvind Krishnamurthy<sup>1</sup> Luis Ceze<sup>12</sup>

## Abstract

由注意力机制驱动的 Transformer 构成了大语言模型 (LLM) 的基础。随着模型规模持续扩大, 高吞吐、低时延推理对高效 GPU 注意力内核的需求变得至关重要。多样化的 LLM 应用也要求注意力方案同时具备灵活性与高性能。我们提出 FlashInfer: 一个面向 LLM 服务、可定制且高效的注意力引擎。FlashInfer 通过块稀疏格式与可组合格式应对 KV-Cache 存储异构性, 从而优化内存访问并减少冗余; 同时提供可定制注意力模板, 借助即时编译 (JIT) 适配不同场景。此外, FlashInfer 的负载均衡调度算法可适应用户请求的动态变化, 并保持与要求静态配置的 CUDA Graph 兼容。FlashInfer 已集成到 SGLang、vLLM 与 MLC-Engine 等主流 LLM 服务框架。全面的内核级与端到端评测表明, FlashInfer 在多种推理场景中均可显著提升内核性能: 相较于现有最先进的 LLM 服务方案, FlashInfer 在 LLM 服务基准中相对编译器后端将 token 间时延降低 29-69%, 在长上下文推理中将时延降低 28-30%, 并在并行生成场景中带来 13-17% 的加速。

项目主页: <http://flashinfer.ai>

## 1 引言

Transformer 架构已成为大语言模型 (LLM) 的核心骨干, 其中最突出的组成部分是注意力机制 (Vaswani et al., 2017)。随着 LLM 快速演进并在各类领域落地, 对高效 GPU 注意力内核的需求持续增长, 目标是在保持可扩展性的同时实现高响应推理。在 LLM 推理中, 注意力计算处于核心位置: 它处理历史上下文, 并基于查询向量生成输出。在服务场景下, 注意力算子会从存储历史信息的 KV cache 中读取数据, 再结合当前 query 完成计算。因此, 注意力算子的效率直接决定了 LLM 推理系统的整体性能。然而, 为 LLM 服务定制高性能注意力内核, 会带来传统训练环境中不常见的挑战。

为 LLM 系统构建高效注意力支持主要面临两类挑战:

**LLM 应用呈现多样化工作负载模式与输入动态性。** LLM 服务涉及多种注意力计算模式, 从上下文处理的 prefill 到在线服务中的批量解码 (Yu et al., 2022)。在多请求并发处理时, 会出现前缀复用机会; 同时, 推测式场景中的树解码会引入额外注意力模式 (Cai et al.,

2024; Miao et al., 2024; Chen et al., 2024)。此外, 同一 batch 内及不同时间步之间的 query 长度与 KV cache 长度都在变化, 朴素实现容易出现负载不均, 因而需要内核具备动态自适应调度能力以达到最优性能。

**现代硬件实现要求注意力算子可定制。** 在内存侧, Paged Attention (Kwon et al., 2023)、Radix Tree (Zheng et al., 2023b) 等高效存储格式对于管理不断增长的 KV cache 规模与多样存储模式至关重要。在计算侧, 为充分释放不同 GPU 架构的性能潜力, 需要构建架构特定的流水线与模板 (Dao, 2023; Shah et al., 2024)。此外, 现代 LLM 中注意力机制的变体也不断增加, 例如分组注意力头 (Ainslie et al., 2023; Shazeer, 2019)、特殊掩码 (Beltagy et al., 2020)、以及定制化的注意力分数计算 (Rivière et al., 2024; xAI, 2023; Ramapuram et al., 2024), 这要求实现策略同时具备灵活性与可扩展性。

工作负载多样性与硬件异构性的叠加, 使得构建统一注意力方案变得复杂。当前各系统往往只覆盖上述特征的一个子集, 导致维护成本高、效率潜力受限。为应对这些挑战, 我们提出 FlashInfer: 一个基于代码生成的注意力引擎, 旨在加速 LLM 中的注意力计算。其核心设计包括:

**FlashInfer 使用块稀疏格式应对 KV-Cache 存储异构性。** 该格式可作为多类 KV-Cache 配置的统一数据结构, 且可调块大小支持细粒度稀疏 (如向量级稀疏) (Chen et al., 2021; Li et al., 2022), 从而统一多样 KV-Cache 模式并提升内存访问效率。

<sup>\*</sup>Part of the work was done while Zihao Ye was interning at NVIDIA. <sup>1</sup>Paul G. Allen School of Computer Science & Engineering, University of Washington <sup>2</sup>NVIDIA <sup>3</sup>Perplexity AI <sup>4</sup>Carnegie Mellon University <sup>5</sup>Independent Researcher. Correspondence to: Zihao Ye <zhye@cs.washington.edu>.

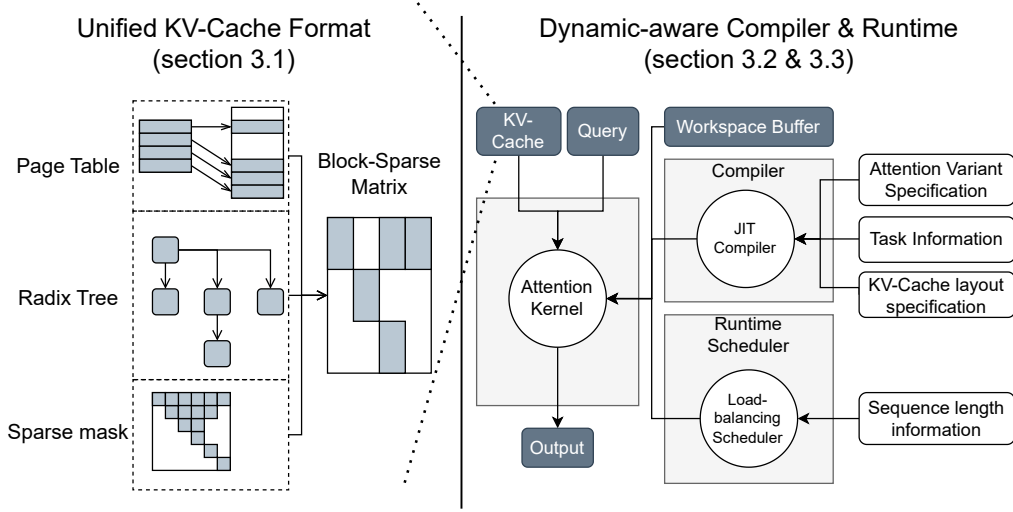


Figure 1. FlashInfer 系统设计概览：注意力变体规格、任务信息与 KV-cache 布局细节在编译期提供以进行 JIT 编译；序列长度信息在运行期输入以进行动态调度。

**FlashInfer** 提供可定制注意力模板以支持不同注意力变体。用户可通过可编程接口实现自身注意力变体；FlashInfer 通过即时编译（JIT）将其转换为高度优化的块稀疏实现，以快速适配不同配置。

**FlashInfer** 采用动态负载均衡调度框架高效处理输入动态性。该框架将编译期 tile 大小选择与运行期调度解耦，提供轻量 API，在推理过程中随 KV-Cache 长度变化自适应调度，同时满足 CUDAGraph 对静态配置的要求 (Gray, 2019; Nguyen et al., 2021)。

图 1 展示了我们的系统设计。我们在标准 LLM 服务场景与前缀共享、推测解码等创新场景下评估了 FlashInfer 性能。FlashInfer 已集成到 vLLM (Kwon et al., 2023)、MLC-Engine (MLC Community, 2024; Lai et al., 2023)、SGLang (Zheng et al., 2023b) 等主流 LLM 服务引擎。评测结果表明，FlashInfer 在标准服务基准以及长上下文推理、并行生成等新场景中均带来显著的端到端时延与吞吐提升。

我们的贡献如下：

- 提出灵活的块稀疏与可组合格式，应对 KV-Cache 存储异构性，实现高效内存管理与访问。
- 开发可定制注意力模板，支持多样注意力变体，并通过 JIT 编译保证高性能执行。
- 设计动态调度框架，在兼容 CUDAGraph 的同时处理输入动态性并最大化硬件利用率。
- 给出全面评估，展示内核级与端到端性能的显著提升。

## 2 背景

### 2.1 FlashAttention

FlashAttention (Dao et al., 2022) 是一种在降低内存开销的同时精确计算注意力的高效算法。在前向传播中，它使用在线 softmax 技巧 (Milakov & Gimelshein, 2018)，在常量级片上内存中在线更新注意力输出，从而避免在 GPU 全局内存中物化完整注意力矩阵。FlashAttention2&3 (Dao, 2023; Shah et al., 2024) 通过针对 Ampere 与 Hopper GPU 优化循环顺序与流水线设计进一步提升性能。FlashInfer 建立在这些进展之上。

FlashAttention 的操作强度为  $O\left(\frac{1}{1/l_{qo}+1/l_{kv}}\right)$ ，其中  $l_{qo}$  与  $l_{kv}$  分别表示 query 长度与 key-value cache 长度。在 LLM 服务中，query 长度通常等于 (prefill) 或小于 (decode/增量 prefill) KV cache 长度，因此可近似简化为  $O(l_{qo})$ 。批处理等技术 (Yu et al., 2022) 并不会改变这一操作强度。多查询注意力 (MQA) (Shazeer, 2019) 与分组查询注意力 (GQA) (Ainslie et al., 2023) 通过将多个 query 组共享同一组 KV 条目来优化 KV-Cache 大小。记 query 头数与 KV 头数比值为组大小  $g = \frac{H_{qo}}{H_{kv}}$ ，则操作强度可提升为  $O(g \cdot l_{qo})$ 。

### 2.2 注意力组合

Block-Parallel Transformer (BPT) (Liu & Abbeel, 2023) 表明：对于同一 query 与不同 key/value 的注意力输出，可以在保留注意力输出与其尺度信息的前提下进行组合。设  $\mathbf{q}$  为一个 query， $\mathcal{I}$  为索引集合。我们将  $\mathcal{I}$  上的注意力尺度定义为注意力分数上的 log-sum-exp：

$$\text{LSE}(\mathcal{I}) = \log \sum_{i \in \mathcal{I}} \exp(\mathbf{q} \cdot \mathbf{k}_i) \quad (1)$$

其中  $\mathbf{k}_i$  为第  $i$  个 key 向量。对应的注意力输出  $\mathbf{O}(\mathcal{I})$  为

$$\mathbf{O}(\mathcal{I}) = \sum_{i \in \mathcal{I}} \frac{\exp(\mathbf{q} \cdot \mathbf{k}_i)}{\exp(\text{LSE}(\mathcal{I}))} \cdot \mathbf{v}_i \quad (2)$$

我们将  $\mathcal{I}$  的注意力状态 (*Attention State*) 定义为注意力输出与注意力尺度组成的二元组:  $\begin{bmatrix} \mathbf{O}(\mathcal{I}) \\ \text{LSE}(\mathcal{I}) \end{bmatrix}$ 。关键在于,  $\mathcal{I} \cup \mathcal{J}$  的注意力状态可由  $\mathcal{I}$  与  $\mathcal{J}$  的状态组合得到。具体地, 引入算子  $\oplus$ :

$$\begin{aligned} \begin{bmatrix} \mathbf{O}(\mathcal{I} \cup \mathcal{J}) \\ \text{LSE}(\mathcal{I} \cup \mathcal{J}) \end{bmatrix} &= \begin{bmatrix} \mathbf{O}(\mathcal{I}) \\ \text{LSE}(\mathcal{I}) \end{bmatrix} \oplus \begin{bmatrix} \mathbf{O}(\mathcal{J}) \\ \text{LSE}(\mathcal{J}) \end{bmatrix} \\ &= \begin{bmatrix} \frac{\exp(\text{LSE}(\mathcal{I}))\mathbf{O}(\mathcal{I}) + \exp(\text{LSE}(\mathcal{J}))\mathbf{O}(\mathcal{J})}{\exp(\text{LSE}(\mathcal{I})) + \exp(\text{LSE}(\mathcal{J}))} \\ \log(\exp(\text{LSE}(\mathcal{I})) + \exp(\text{LSE}(\mathcal{J}))) \end{bmatrix} \end{aligned}$$

由于  $\oplus$  具有结合律与交换律, 多个注意力状态可按任意顺序组合。Ring-Attention (Liu et al., 2023) 与 Flash-Decoding (Dao et al., 2023) 利用这一性质卸载部分注意力计算, 以降低内存开销并提升硬件效率。在 FlashInfer 中, 注意力状态被采用为注意力操作的规范输出,  $\oplus$  则作为这些状态上的标准归约算子 (类似 GEMM 中的求和)。

### 2.3 块/向量稀疏

Block Compressed Sparse Row (BSR) 是一种硬件友好的稀疏格式: 它将非零元素组织为大小为  $(b_r, b_c)$  的连续子矩阵, 而不是非结构化稀疏中的随机分散模式。相较标准 CSR, BSR 具有多项优势: 可提升寄存器复用效率 (Im et al., 2004; Buluç et al., 2009), 并与 GPU/NPU 上的硬件矩阵乘单元有更好兼容性 (Narang et al., 2017; Gray et al., 2017); 此外还能跳过空块, 降低计算开销。当子计算与硬件矩阵乘指令 (如 NVIDIA 的 `mma`) 对齐时, BSR 的效率优势尤为明显。传统上, Tensor Core 指令的最小维度通常为 16 (新架构更大), 因此多数块稀疏内核使用 (16, 16) 的倍数作为块大小。但这对细粒度稀疏场景并非总是最优 (Wang et al., 2023)。很多注意力库将块稀疏注意力的块大小限制在 (128, 128) 的倍数。

近期研究 (Chen et al., 2021; Li et al., 2022) 表明, 即便在更小块大小下也能高效利用 Tensor Core, 例如 GEMM 中矩阵  $B$  采用 (16, 1), 或矩阵  $A$  采用 (1, 16) (即向量稀疏)。其方法是先将行/列 gather 到连续共享内存, 再在该连续数据上执行稠密 Tensor Core 计算。这对细粒度稀疏应用尤为有利。FlashInfer 在此基础上进一步支持任意列块大小  $B_c$ , 从而在多样稀疏模式下获得更高灵活性与效率。

## 3 设计

本节介绍 FlashInfer 的系统设计。我们首先给出 FlashInfer 使用的数据结构, 并说明块稀疏行 (BSR) 如何成为注意力内核中 KV cache 存储的通用抽象。随后介绍 FlashInfer 编译器: 它支持多类注意力变体, 并配合一个感知动态输入的运行时调度器, 实现注意力内核的负载均衡调度。最后, 我们描述面向用户的 API, 以便将 FlashInfer 集成到现有 LLM 服务系统中。

### 3.1 KV-Cache 存储

#### 3.1.1 将块稀疏矩阵作为统一格式

近期 KV-Cache 存储方案 (如 PageAttention (Kwon et al., 2023)、RadixAttention (Zheng et al., 2023b)) 通常采用非连续内存布局, 其最小粒度为一个  $(H, D)$  的张量块 (或 token), 其中  $H$  是头数,  $D$  是隐藏维度。这些结构旨在最小化内存碎片, 同时提高复用率与缓存命中率。我们表明, 这些异构数据结构都可在块稀疏格式下统一表示, 如图 2 所示。

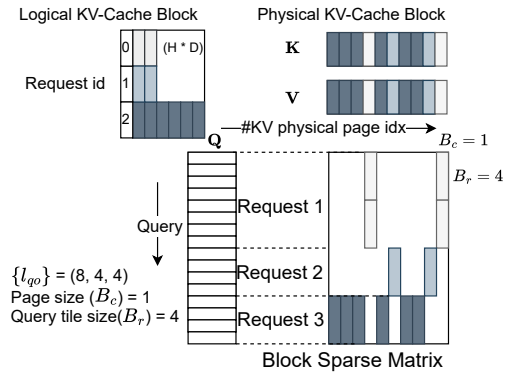


Figure 2. 将页表表示为 BSR ( $B_r = 4, B_c = 1$ ) 格式。块稀疏矩阵的列块数对应页表已分配块的总数; 非零块表示 query 会访问到的 KV-Cache 页面。

将页表与稀疏矩阵在概念上等价的思路在 SP-Grid (Setaluri et al., 2014) 中已有探索, 其借助 TLB 硬件高效索引稀疏结构。除页表与基数树外, 稀疏矩阵也能表达多种注意力机制, 例如推测解码中的树注意力 (Cai et al., 2024; Miao et al., 2024; Chen et al., 2024), 以及施加在 KV-Cache 上的重要性掩码 (Tang et al., 2024)。

在 FlashInfer 中, 我们采用统一数据表示策略。query 与 output 使用无 padding 的 ragged tensor (又称 jagged array) (Tensorflow Developers, 2018) 存储, 便于将不同请求的 query/output 紧凑打包到同一张量。初始时, key/value 也使用与 query 相同索引指针的 ragged tensor (因为它们来自同一输入经  $W_q, W_k, W_v$  投影)。随后, 这些 key/value 会写入 KV-Cache 并更新



索引。KV-Cache 采用 BSR 格式，块大小由应用决定： $B_r$  对应 query tile 大小（后文介绍）， $B_c$  由 KV-Cache 管理算法给出。FlashInfer 内核支持任意  $(B_r, B_c)$ 。

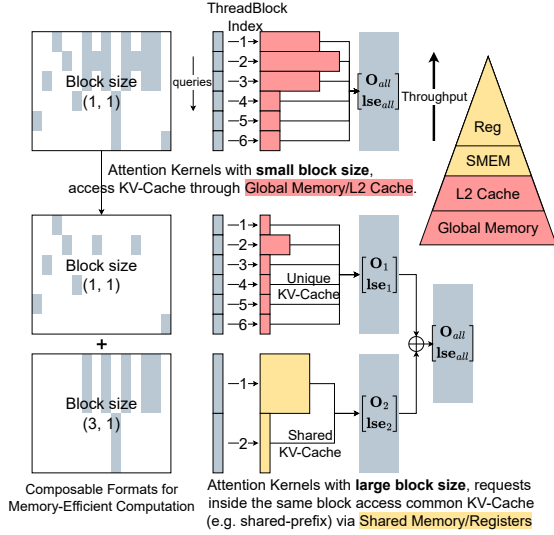


Figure 3. 用于共享前缀分解的可组合格式。前 6 行 query 共享前缀，后 6 行 query 也共享前缀。我们用块大小 (3,1) 的块稀疏矩阵存储共享前缀对应的 KV cache，用块大小 (1,1) 的块稀疏矩阵存储其余非共享 KV cache。对于 (3,1)，3 个 query 可在高带宽共享内存中复用同一 KV cache；对于 (1,1)，每个 query 只能在各自 threadblock 内访问 KV cache，路径仅能走低带宽全局内存或 L2。

### 3.1.2 用于内存效率的可组合格式

受 SparseTIR (Ye et al., 2023) 启发，我们通过可组合格式提升注意力计算效率。其核心是使用多个块稀疏格式共同存储一个稀疏矩阵，而非单一格式，从而在灵活性与内存效率上更优。单一块稀疏格式受固定块大小约束，其内存效率受块行数  $B_r$  限制：较大  $B_r$  可提升同块请求的共享内存与寄存器复用，但也会增大碎片；不同块之间又无法共享彼此的片上数据。

我们的可组合格式允许基于先验知识对 KV cache 稀疏矩阵进行分解。例如，若部分请求共享前缀，其对应行列会形成稠密子矩阵，此时可用更大  $B_r$  的块稀疏矩阵高效表示。图 3 展示了这一思路。该方法无需移动 KV cache 数据，只需计算稀疏子矩阵的索引与索引指针数组。对较大块大小执行注意力计算时，可通过高速共享内存与寄存器复用共享 KV cache，显著提升内存效率。

## 3.2 计算抽象

我们基于 CUDA/CUTLASS (Thakkar et al., 2023) 开发了 FlashAttention 模板，面向稠密与块稀疏矩阵，并兼容 Turing 到 Hopper (sm75 到 sm90a) 的 NVIDIA GPU。对于 Ada (sm89) 及以下架构，我们

采用 FlashAttention2 (简称 FA2) 算法 (Dao, 2023)；对 Hopper，采用 FlashAttention3 (简称 FA3) 算法 (Shah et al., 2024)。关键改进包括：更高效的稀疏 tile 共享内存加载、更丰富的 tile 大小配置、针对 GQA 的优化内存访问模式，以及可定制注意力变体支持。

### 3.2.1 从全局内存到共享内存的数据搬运

FlashInfer 的注意力模板支持任意块大小，因此需要专门的数据加载机制，因为块形状可能与 Tensor Core 形状不对齐。正如第 2.3 节所述，我们通过将分散的全局内存 tile 搬运到连续共享内存，再执行稠密 Tensor Core 计算来解决该问题。一次 MMA 指令所需输入可能来自块稀疏矩阵中的不同块。图 4 展示了 FlashInfer 如何将稀疏/稠密 KV-Cache 的 tile 载入共享内存：稀疏 KV-Cache 地址通过 BSR 的 indices 数组计算，稠密地址通过行索引仿射变换计算。

KV-Cache 的最后一维保持连续 (head dimension  $d$ ，通常 128 或 256)，从而维持与 GPU cache line 匹配的合并访存。我们采用 128B 宽度的异步拷贝指令 LDGSTS 以最大化内存带宽利用。尽管 Hopper 的 TMA (Tensor Memory Accelerator) 可进一步加速数据搬运，但其不支持非仿射访问模式，因此仅在连续 KV-Cache 的 Hopper 场景使用 TMA；其他不适配场景回退至 Ampere 风格异步拷贝。

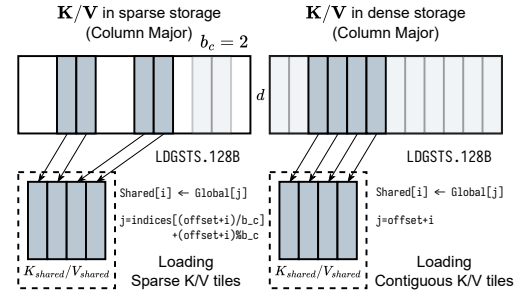


Figure 4. FlashInfer 中稀疏/稠密 KV-Cache 从全局内存到共享内存的数据搬运。左：稀疏 KV-Cache， $b_c = 2$ ；右：稠密 KV-Cache。head dimension 为  $d$ 。

搬运到共享内存后，稀疏与稠密 FlashAttention 实现将汇合：仅数据加载模块不同，核心计算内核可统一复用。

### 3.2.2 多 tile 大小微内核

为适配 LLM 应用中不断变化的操作强度，FlashInfer 在多种 tile 大小上实现了 FA2。传统 FA2 常使用少量固定 tile (如 (128, 64))，在 A100 prefill 上效果较好，但对短 query 的解码场景效率不足。某一架构的最优 tile 在另一架构不一定合适；例如 Ada (sm89) 共享内存较小，大 tile 会影响 SM 占用。

FlashInfer 提供  $\{1, 16, 32, 64, 128\} \times \{32, 64, 128\}$  的

FA2 内核，并基于硬件资源与工作负载强度使用启发式选择：

1. 计算 batch 平均 query 长度 (对 GQA, 会将 query 长度与 head group 维融合, 见附录 A), 并选择不小于该长度的最小 query tile。
2. 将寄存器与共享内存约束表示为 K/V tile 大小的函数, 在约束下最大化 SM 资源占用。

### 3.2.3 面向注意力变体的 JIT 编译器

当 query tile 为 1 时, 我们使用 CUDA Core 模板 (因为 Tensor Core 指令  $m$  维最小为 16); 其余 query tile 大小使用 Tensor Core。对于 FA3, 我们提供行 tile 为 64 倍数的配置以匹配 Hopper 的 WGMMA 要求。tile 大小在编译期结合任务类型 (解码、prefill 等) 和硬件能力确定。块稀疏矩阵的块行大小  $B_r$  与 query tile 大小  $T_q$  对齐。

近年的 LLM 广泛使用标准注意力的变体。若在 CUDA 库中逐一内建支持并针对每个变体专门优化内核, 维护不可持续, 而变体数量仍在快速增加。另一方面, 大多数变体与标准注意力结构相近, 因此可复用 FlashAttention 内核骨架, 仅做小范围修改。受 FlexAttention (He et al., 2024) 启发, 我们设计了可定制 CUDA 模板与 JIT 编译器: 输入注意力变体规格后, 自动生成优化内核代码。变体规格包含以下函子:

- QueryTransform, KeyTransform, ValueTransform: 注意力计算前施加在 query/key/value 上的变换。
- OutputTransform: 返回前施加在注意力输出上的变换。
- LogitsTransform, LogitsMask: softmax 前施加在 logits 上的变换与掩码。

每个函子都有固定签名: 输入为内核参数、输入张量及当前 query/key/head 索引, 输出为变换结果。这些变体函数由用户定义的变体类成员给出, 从而为函子创建闭包。LogitsTransform/LogitsMask 设计借鉴 FlexAttention (He et al., 2024), 可用于实现定制掩码、logits soft-cap (Rivière et al., 2024; xAI, 2023)、滑窗注意力 (Beltagy et al., 2020) 等变体。FlashInfer 在变体规格中可选择是否使用 softmax, 因此也支持不使用 softmax 的注意力变体 (如 FlashSigmoid (Ramapuram et al., 2024))。FlashInfer 的 query/key 变换函子还能将归一化、RoPE (Su et al., 2024) 与投影 (DeepSeek-AI et al., 2024) 融入注意力内核。

图 5 展示了 FlashInfer 如何将 FlashSigmoid (Ramapuram et al., 2024) 的注意力规格映射到 CUDA 模板的不同位置。注意力规格可直接接受一段 CUDA 代码定义变体函子, 使用户能够使用高级 PTX 指令<sup>1</sup> 甚至自定义库。JIT 编译器通过将变体类及其他信息插入

<sup>1</sup><https://docs.nvidia.com/cuda/parallel-thread-execution/>

### Algorithm 1 FlashInfer 的负载均衡调度算法

---

```

1: Input:  $\{l_{qo}(i), l_{kv}(i)\}_i$ , query tile 大小  $T_q$ 。
2: 定义 tile  $l_q, l_{kv}$  的代价 ( $\alpha, \beta$  为超参数):

$$\text{cost}(l_q, l_{kv}) = \alpha l_q + \beta l_{kv}$$

3: 计算最大 KV chunk 大小  $L_{kv}$ :

$$L_{kv} \leftarrow \frac{\sum_i \lceil \frac{l_{qo}(i)}{T_q} \rceil \cdot l_{kv}(i)}{\#CTA}$$

4: 将每个 query tile 对应的 KV 按最大长度  $L_{kv}$  切分为多个 chunk, 并为每个 chunk 分配工作索引  $w$ , 其长度记为  $l_{kv}(w)$ 。
5: 令  $W = \{(w, l_{kv}(w))\}$ , 并按长度降序排序。
6:  $Q \leftarrow \text{PriorityQueue}(\{(c, 0)\})$ , 其中  $c$  是 CTA 索引。
7: while  $W \neq \emptyset$  do
8:    $c, \text{current\_cost} \leftarrow Q.\text{pop}_{\min}()$ 
9:    $w, l_{kv}(w) \leftarrow W.\text{pop}()$ 
10:   $\text{new\_cost} \leftarrow \text{current\_cost} + \text{cost}(T_q, l_{kv}(w))$ 
11:  将 chunk  $w$  分配给 CTA  $c$ 
12:   $Q.\text{push}((c, \text{new\_cost}))$ 
13: end while

```

---

模板生成 CUDA 代码, 再用 PyTorch JIT 编译器<sup>2</sup> 编译, 并注册为自定义算子<sup>3</sup>。我们也支持通过框架无关的 DLPack 接口<sup>4</sup> 编译到其他运行时系统。

### 3.3 面向动态性的运行时

本节介绍 FlashInfer 的运行时设计, 包括动态调度框架以及面向内存效率的可组合格式。

#### 3.3.1 负载均衡调度

在 FlashInfer 中, 负载均衡调度算法的目标是将工作负载均匀分配到所有 SM, 以最小化 SM 空闲时间。算法输入为 query/output 维与 key/value 维的序列长度信息, 输出包括: 工作负载到 CTA (Cooperative Thread Array) 的映射, 以及部分输出与最终输出之间的索引映射。算法见算法 1 (为简洁起见省略 head 维)。我们的方法受 Stream-K (Osama et al., 2023) 启发; 但由于 LLM 服务需要确定性输出, 我们没有采用 Stream-K 中的原子聚合, 以避免非确定性。给定相同序列长度信息时, 调度算法会生成确定性的聚合顺序。

图 6 展示了 FlashInfer 运行时调度器的工作流程。注意力内核不会直接产出最终输出, 因为较长 KV 会被切分为多个 chunk, 最终输出是所有 chunk 部分输出的收缩 (使用第 2.2 节的注意力组合算子)。部分输出存储在用户提供的 workspace buffer 中 (见第 3.4 节)。FlashInfer 实现了可处理变长聚合的高效注意力组合算子。每个 CTA 的工作队列, 以及部分输出与最终输出之间的映射都由调度器规划。CPU 端生成计划信息后, FlashInfer 会异步拷贝到 GPU workspace

<sup>2</sup>[https://pytorch.org/tutorials/advanced/cpp\\_extension.html#jit-compiling-extensions](https://pytorch.org/tutorials/advanced/cpp_extension.html#jit-compiling-extensions)

<sup>3</sup>[https://pytorch.org/tutorials/advanced/custom\\_ops\\_landing\\_page.html](https://pytorch.org/tutorials/advanced/custom_ops_landing_page.html)

<sup>4</sup><https://github.com/dmlc/dlpack>

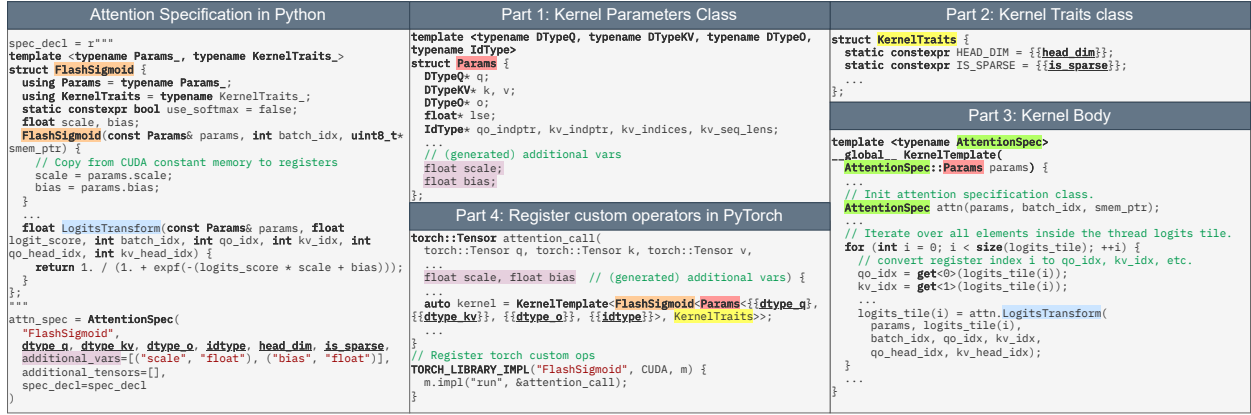


Figure 5. FlashInfer 的注意力变体 JIT 编译器：变体函子、附加变量/张量与数据类型由 CUDA 代码字符串定义，再用于填充内核模板；图中对应代码采用共享高亮。

buffer 的指定区域，作为持久化 attention/contraction 内核输入。随着每个生成步的序列长度变化，调度器在 CPU 端逐步生成计划；该开销可在多层之间摊销，因为同一计划可复用于所有层。

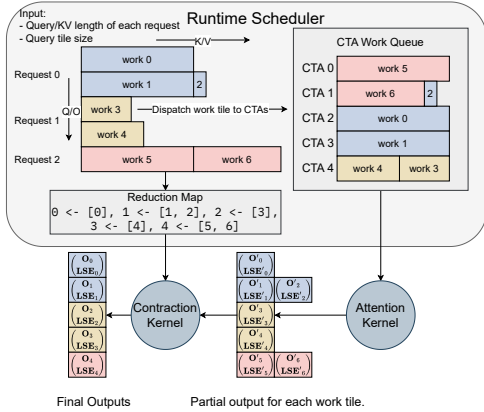


Figure 6. FlashInfer 的负载均衡运行时调度器。query/output 与 key/value 两个维度的序列长度输入调度器后，生成计划信息：(1) 每个 CTA 的工作队列；(2) 部分输出与最终输出的索引映射。这些计划信息缓存于 GPU 侧，并作为持久化 attention/contraction 内核输入。

FlashInfer 保证注意力阶段与收缩阶段均兼容 CUDA-Graph (Gray, 2019; Nguyen et al., 2021)。两个阶段都使用持久化内核，且 grid size 在编译后固定，即每个生成步都以相同 grid 启动。我们为 workspace buffer 的每个 section 设定固定偏移，用于存放部分输出与计划信息，确保每个生成步传入内核的指针保持一致，满足 CUDA-Graph 要求 (详见附录 D.1)。我们还将两个阶段合并为一个持久化内核，消除内核间额外开销。

### 3.4 编程接口

FlashInfer 提供了面向集成的编程接口，可无缝接入 vLLM(Kwon et al., 2023)、MLC-Engine(MLC Community, 2024)、SGLang (Zheng et al., 2023b) 等现有 LLM 服务框架。

```
# Create workspace buffer
workspace = torch.empty(...)
seqlen_info.init()

# Compile: create CUDA-Graphs
graphs = []
for task_info in task_infos:
    # Init: compile kernels according to spec
    attn = AttentionWrapper(attn_spec, task_info, workspace)
    g = torch.cuda.CUDA-Graph()
    # Dummy plan
    attn.plan(seqlen_info)
    # Capture CUDA graphs
    with torch.cuda.graph(g):
        for i, layer in enumerate(layers):
            ...
            attn.run(...)
            ...
    graphs.append(g)

# Runtime: select the best CUDA-Graph
g = select_graph(graphs)
finished = False
# Text generation loop
while not finished:
    seqlen_info.update()
    # Plan per generation step
    attn.plan(seqlen_info)
    # Replay CUDA-Graph
    g.replay()
```

Listing 1. FlashInfer 的 PyTorch 编程接口

代码清单 1 展示了 FlashInfer 的 PyTorch 接口。用户在初始化 wrapper 时提供注意力变体规格、任务信息和用户分配的 workspace buffer (详见附录 D)，用于存储 FlashInfer 动态调度所需的部分输出与计划信息。内核在初始化时 JIT 编译并缓存复用。对于可组合格式 (第 3.1.2 节)，FlashInfer 会创建多个注意力 wrapper，每个 wrapper 使用不同块大小。不同平



均 query 长度与可组合格式配置对应的内核会分别编译，并捕获到不同 CUDAGraph 中。运行时，服务框架根据当前 KV-Cache 配置选择最合适的 CUDAGraph，从而在不同工作负载下保持最优性能。

`plan` 函数通过处理序列长度信息生成负载均衡调度计划，从而激活动态调度器。计划可缓存并在序列长度规格一致的算子间复用，例如同一生成步中的所有 decode attention。`run` 函数使用 query、key、value 与缓存计划执行注意力计算并输出结果。CUDAGraph 可以捕获 `run` 调用并将整个注意力生成步骤编译为单图；但 `plan` 位于 CPU 侧，因此不会被 CUDAGraph 捕获。`plan/run` 的拆分受 Inspector-Executor (IE) 模型 (Mirchandaney et al., 1988; Saltz & Mirchandaney, 1991; Ponnusamy et al., 1993) 启发，该模型被广泛用于并行化不规则工作负载。

## 4 评估

本节从内核级与端到端两个层面对 FlashInfer v0.2 进行评估，展示其设计如何应对 LLM 服务中的关键挑战。结果表明：在 LLM 服务基准中，相比 Triton 后端，FlashInfer 可将 token 间时延降低 29-69%；在长上下文推理中可降低 28-30% 时延；在并行生成中可获得 13-17% 加速。实验在 NVIDIA A100 40GB SXM 与 H100 80GB SXM 上进行，使用 CUDA 12.4、PyTorch 2.4.0，存储与计算精度为 f16。

### 4.1 端到端 LLM 服务性能

我们在 SGLang v0.3.4 (Zheng et al., 2023b) 上评估 FlashInfer，并与 SGLang + Triton v3.0 (Tillet et al., 2019) 对比。后者是面向 NVIDIA GPU 的领先 LLM 服务引擎，但其注意力内核闭源，限制了透明性与社区优化潜力。为保证评估全面性，我们使用两类数据集：常用 ShareGPT 数据集<sup>5</sup>与合成负载 (Variable，序列长度在 512 到 2048 间均匀分布)。我们在时延敏感在线服务设置下测量 TTFT (time-to-first-token) 与 ITL (inter-token-latency)，并调整请求率使 P99 TTFT 保持在 200ms 以下。

图 7 展示了在 Llama 3.1 8B (1xH100) 与 Llama 3.1 70B (4xH100) 上的 ITL 与 TTFT。相较 Triton 后端，FlashInfer 后端在所有设置下都表现出稳定加速。

### 4.2 输入动态性下的内核性能

本节在不同序列长度分布下，对比 FlashInfer 生成内核与开源 SOTA FlashAttention 库性能。我们使用其最新主分支<sup>6</sup> (包含 FlashAttention2 与 FlashAttention3)。

<sup>5</sup>[https://huggingface.co/datasets/anon8231489123/ShareGPT\\_Vicuna\\_unfiltered/resolve/main/ShareGPT\\_V3\\_unfiltered\\_cleaned\\_split.json](https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered/resolve/main/ShareGPT_V3_unfiltered_cleaned_split.json)

<sup>6</sup>Commit: c1d146c

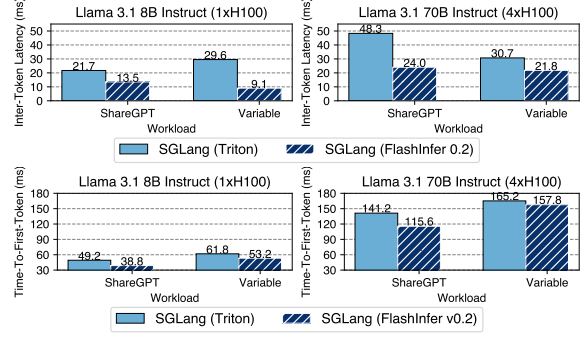


Figure 7. 集成 FlashInfer 与 Triton 的 SGLang 的中位 ITL 与中位 TTFT。

固定 batch size 为 16，并测试三种长度分布：常数 (1024)、均匀分布 (512 到 1024) 与偏斜分布 (均值 1024 的 Zipf)。prefill 内核启用因果掩码，这是 LLM 服务的常见设置。

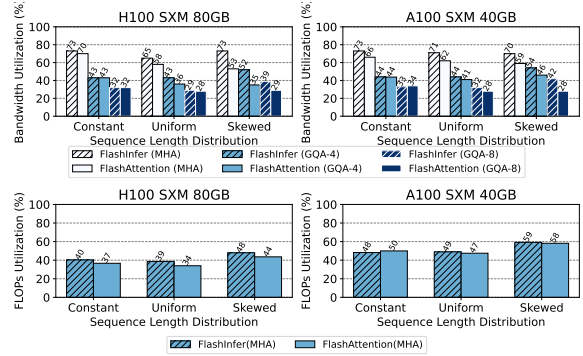


Figure 8. decode (上) 与 prefill (下) 内核的带宽利用率与 FLOPs 利用率 (越高越好)。

图 8 给出了 decode 与 prefill 内核的带宽/FLOPs 利用率。由于我们的动态负载均衡调度器 (第 3.3.1 节)，FlashInfer 在均匀与偏斜长度分布下显著优于 FlashAttention。FlashInfer 在 decode 场景也优于 FlashAttention，原因在于我们支持更灵活的 tile 大小选择 (第 3.2.2 节)，而 FlashAttention 在解码场景使用了次优 tile 配置。

### 4.3 面向长上下文推理的可定制性

本节展示 FlashInfer 的定制化注意力内核如何显著加速 LLM 推理。我们关注 Streaming-LLM (Xiao et al., 2023)，该算法可在常量 GPU 内存下实现百万 token 推理。Streaming-LLM 对专用注意力内核有较强需求，特别是 RoPE (Su et al., 2024) 与注意力融合内核；FlashInfer 仅需额外约 20 行 query/key 变换代码即可生成此类融合内核。我们对比 FlashInfer 融合内核与非融合内核 (FlashInfer 与 FlashAttention)，并量化 FlashInfer 集成后端到端时延收益。

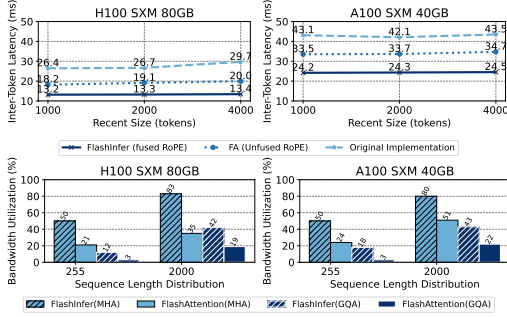


Figure 9. 上：Streaming-LLM 使用 FlashInfer 融合内核与 FlashAttention 非融合内核的端到端时延（含原始实现）。下：FlashInfer 融合 RoPE 内核相对 FlashAttention 非融合内核的带宽利用率。

端到端实验中，我们在 MT-Bench (Zheng et al., 2023a) 上运行 Vicuna-13B (Chiang et al., 2023) 推理，并测量启用/禁用 FlashInfer 内核时的 ITL。图 9 显示，在我们优化后的 Streaming-LLM 实现中（原实现存在额外开销），FlashInfer 融合内核在不同配置（通过改变 recent window size）下可带来 28-30% 的时延下降。我们同时给出 FlashInfer 融合 RoPE 内核与“FlashAttention 的 RoPE + FlashAttention 注意力”组合的内核级对比。FlashInfer 融合内核的带宽利用率提升达 1.6-3.7x，说明可定制注意力内核非常关键。

#### 4.4 并行生成性能

本节说明 FlashInfer 的可组合格式如何提升并行解码。并行生成已成为 LLM 服务的重要任务，在 LLM agent 场景中尤为有用。OpenAI API 通过 “n” 参数<sup>7</sup> 支持一次生成多个候选。由于常存在共享前缀，prefix-caching 可显著提升并行生成效率。FlashInfer 的可组合格式（第 3.1.2 节）可将共享前缀与后缀的注意力计算解耦，从而加速并行解码。

我们在启用 prefix-caching 的 MLC-Engine (MLC Community, 2024) 中实现可组合格式，并评估并行生成性能。实验使用 Llama 3.1 8B/70B (Dubey et al., 2024) 与 ShareGPT，固定请求率为 16，并将并行 token 数设置为 {1, 2, 4, 8, 16, 32, 64}，对比关闭可组合格式的配置。图 10 给出了启用/禁用可组合格式时的 ITL 与 TTFT。

在中等并行度 ( $4 \leq n \leq 32$ ) 下，FlashInfer 的可组合格式在 ITL 与 TTFT 上均带来稳定加速。峰值出现在  $n = 4$ ：8B/70B 的 ITL 分别下降 13.73%/17.42%，TTFT 分别下降 16.41%/22.86%。较小  $n$  下，由于块大小提升不足，收益有限；较大  $n$  下，计算不再主要由注意力主导（尤其 ShareGPT 序列较短），可组合格式优势趋于平台期。

<sup>7</sup><https://platform.openai.com/docs/api-reference/chat/create>

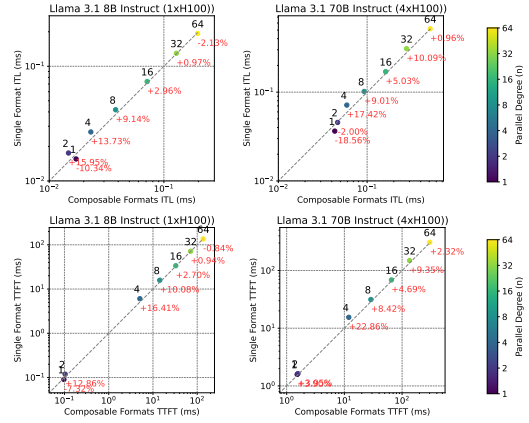


Figure 10. MLC-Engine 在并行生成下启用/禁用可组合格式的 ITL 与 TTFT。横轴表示可组合格式性能，纵轴表示单格式性能；点在对角线上方表示可组合格式更优。不同颜色对应不同并行生成  $n$ 。

## 5 相关工作

### 5.1 注意力优化

多头注意力 (MHA) (Vaswani et al., 2017) 面临计算与 IO 双重挑战。FasterTransformer (NVIDIA, 2021) 通过融合多头注意力 (FMHA) 降低全局内存占用，但其共享内存开销随序列长度线性增长，因此难以扩展到长上下文。ByteTransformer (Zhai et al., 2023) 则针对变长输入优化了 FMHA。FlashAttention (Dao et al., 2022) 采用在线 softmax (Milakov & Gimelshein, 2018) 将共享内存占用降为常量，从而支持长上下文。FlashAttention2&3 (Dao, 2023; Shah et al., 2024) 进一步通过改进循环结构、重叠 softmax 与 GEMM 来提升性能。FlashDecoding (Dao et al., 2023) 将 Split-K 用于解码注意力内核。LeanAttention (Sanovar et al., 2024) 使用 StreamK (Osama et al., 2023) 缓解固定长度注意力中的 wave-quantization (NVIDIA, 2023b)。FlashInfer 在 FlashAttention2&3 模板上扩展以支持稀疏注意力，并在变长序列场景下采用类 StreamK 优化。Nanoflow (Zhu et al., 2024a) 引入 GEMM、注意力与通信的横向融合，POD-Attention (Kamath et al., 2024) 则专注优化 chunked-prefill 注意力。FlashInfer 的 JIT 框架可扩展生成支持这类融合的内核。FlashDecoding++ (Hong et al., 2024) 利用注意力尺度统计预定义统一最大值，将注意力组合（第 2.2 节）转化为求和，从而可通过 TMA Store Reduce (Colfax, 2024) 异步更新全局 attention state。该方向与 FlashInfer 贡献正交，我们将其留作未来工作。

RelayAttention (Zhu et al., 2024b)、Hydragen (Juravsky et al., 2024)、ChunkAttention (Ye et al., 2024)、Parrot (Lin et al., 2024) 等工作探索了共享前缀解码注意力，但通常需要为前缀与后缀维护独立 KV-Cache。相比之下，FlashInfer 的可组合格式支持多层级、多前缀解码，并统一页表管理，因此可在不改动内存管理



模块的情况下无缝集成到 LLM 服务框架。

## 5.2 GPU 上的稀疏优化

FusedMM (Rahman et al., 2021) 探索了稀疏-稠密矩阵乘 (SpMM) 融合, 但未覆盖 softmax 计算, 难以直接用于注意力加速。Zhang et al. (2022) 研究图注意力网络 (GAT) 内核融合, SAR (Mostafa, 2022) 则序列化稀疏注意力聚合, 思路类似 FlashAttention, 但二者都未探索 Tensor Core 使用。Blocksparse (Gray et al., 2017) 通过 BSR GEMM 利用 Tensor Core。Chen et al. (2021)、TC-GNN (Wang et al., 2023) 与 Magicube (Li et al., 2022) 提出向量稀疏格式以高效利用 Tensor Core。FlashInfer 在此基础上进一步支持 FlashAttention 下任意  $(b_r, b_c)$  块大小。

## 5.3 注意力编译器

FlexAttention (He et al., 2024) 提供用户友好的注意力变体编程接口, 并将其编译为基于 Triton (Tillet et al., 2019) 的块稀疏 flashattention; 其使用 PyTorch Compiler (Ansel et al., 2024) 自动生成反向传播。FlashInfer 在此基础上扩展了接口以支持 query/key 变换, 并聚焦 LLM 服务场景下的向量稀疏与负载均衡。由于 Triton 在不少场景仍落后于 CUDA & CUTLASS, FlashInfer 选择生成 CUDA 代码而非 Triton。FlashInfer 也可作为 FlexAttention 前向阶段的后端。Mirage (Wu et al., 2024) 通过概率等价验证器优化 GEMM 与 FlashAttention 的 tiling 策略, 依赖 Triton/CUTLASS 生成代码; 但其不支持变长与稀疏数据结构, 也不包含 safe-softmax。相比之下, FlashInfer 可直接用于 LLM 服务。

## 5.4 LLM 服务系统

Orca (Yu et al., 2022) 引入连续批处理以提升吞吐。PagedAttention (Kwon et al., 2023) 使用页表管理 KV-Cache。Sarathi-serve (Agrawal et al., 2024) 通过将 decode 与 chunked-prefill 搭配执行提升效率, SGLang (Zheng et al., 2023b) 则用 RadixTree 提升前缀缓存与 KV 管理。FlashInfer 通过块稀疏注意力内核为这些机制提供统一支持。vAttention (Prabhu et al., 2024) 证明 GPU 虚拟内存可在无需专用内核情况下处理 PageAttention 的地址转换; 但动态 KV-Cache 稀疏等挑战依然存在 (例如 Quest (Tang et al., 2024)), 此时 FlashInfer 的块稀疏内核仍然有效。此外, FlashInfer 也可与 vAttention 结合, 通过生成连续 KV-Cache 内核来协同工作。

## 6 讨论

当前 FlashInfer 仅支持注意力计算的前向传播。若要将 FlashInfer 扩展到训练场景, 需要开发可定制的反向注意力内核模板, 这是我们未来计划探索的方向。就 FlashInfer 的通用性而言, 我们将计算

与 tile 调度解耦, 使得 FlashDecoding (Hong et al., 2024)、Lean Attention (Sanovar et al., 2024) 等不同分块策略都可通过运行时调度器实现, 而无需将调度逻辑硬编码到注意力模板内部。这种设计将调度算法 (见算法 1) 泛化为统一框架, 可同时处理负载均衡与 wave-quantization 缓解, 并在不同架构上追求最优 GPU 性能 (例如 Turing/Ampere/Ada 的 FlashAttention2 (Dao, 2023) 与 Hopper 的 FlashAttention3 (Shah et al., 2024))。尽管 Triton (Tillet et al., 2019) 等框架提供了 GPU 无关接口, 但其对新硬件特性的支持往往滞后; 我们的模板设计空间写作  $f_{\text{epilogue}}(\text{scan}(f_{\text{logits}}(f_q(Q) \cdot f_k(K))) \cdot f_v(V))$ , 可覆盖大多数注意力函数, 包括近期的 Multi-head Latent Attention (MLA) (DeepSeek-AI et al., 2024) 与线性注意力中的 intra-attention 组件 (Yang et al., 2024)。

## 7 结论与未来工作

本文提出了 FlashInfer: 一个面向 LLM 服务、灵活且高效的注意力引擎。我们提出统一的块稀疏存储与可组合格式以提升内存效率, 使用 JIT 编译实现可定制性, 并引入负载均衡调度器应对输入动态性。我们在多种推理场景下评估了 FlashInfer, 结果显示其在内核级与端到端 LLM 服务指标上均具有强性能优势。未来我们计划探索将更高层 DSL (Wu et al., 2024; He et al., 2024) 编译为 FlashInfer 的注意力规格, 并支持向其他后端进行代码生成 (Ozen, 2024; Spector et al., 2024; Tillet et al., 2019)。FlashInfer 已开源并可在 <https://github.com/flashinfer-ai/flashinfer> 获取, 且已在生产级系统中规模化部署。

## 8 致谢

我们感谢 MLSys 匿名审稿人的建设性意见, 感谢 LM-SYS ORG、UW Syslab 与 SAMPL 研究组、CMU Catalyst 组的宝贵反馈与讨论; 感谢 Yaxing Cai、Junru Shao、Lianmin Zheng、Ying Sheng、Liangsheng Yin、Lily Liu、Woosuk Kwon、Cody Yu、Ray Wan、Bowen Wang、Pavani Majety、Elfie Guo、Travis Addair、Cuimi Guo 在将 FlashInfer 集成到 LLM 服务框架中的帮助; 感谢 Zhuoming Chen、Lesheng Jin、Antoni Baum、Kaichao You、Simon Mo、Ke Bao、Byron Hsu、Zhiqiang Xie、Haofeng Huang、Sirui Lu、Henry Xiao、Chi-Chih Chang、Yilong Zhao、Size Zheng、Bohan Hou、Yang Yu、Nandor Licker、Tsu Bin、Hieu Pham、Horace He、Vijay Thakkar、Yuxian Qiu、Freddy Qi、June Yang、Bing Xu、Anxhelo Xhebraj、Evghenii Gaburov、Bastian Hagedorn 及所有社区贡献者对 FlashInfer 项目的意见与支持。本工作部分由 NSF 奖项 CNS-2211882 资助, 并获得 OctoAI、Qualcomm 与 CMU 开源软件 fellowship 的捐赠支持。UW 的研究者部分受 NSF 奖项 CCF-1518703 资助, 并受 JUMP 2.0 七个中心中的 ACE 与 PRISM 支持; JUMP 2.0 为半导体研究公司 (SRC) 项目, 由 DARPA 赞助。Zihao Ye 受 NVIDIA

研究生奖学金支持。Luis Ceze 受 Lazowska Endowed Professorship 支持。本文观点与结论不代表上述资助机构立场。

## References

- Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. Taming throughput-latency tradeoff in llm inference with sarathi-serve. *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation, 2024, Santa Clara, 2024*.
- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: training generalized multi-query transformer models from multi-head checkpoints. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pp. 4895–4901. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.EMNLP-MAIN.298. URL <https://doi.org/10.18653/v1/2023.emnlp-main.298>.
- Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., Hirsh, B., Huang, S., Kalambarkar, K., Kirsch, L., Lazos, M., Lezcano, M., Liang, Y., Liang, J., Lu, Y., Luk, C. K., Maher, B., Pan, Y., Puhersch, C., Reso, M., Saroufim, M., Siraichi, M. Y., Suk, H., Zhang, S., Suo, M., Tillet, P., Zhao, X., Wang, E., Zhou, K., Zou, R., Wang, X., Mathews, A., Wen, W., Chanan, G., Wu, P., and Chintala, S. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '24*, pp. 929–947, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400703850. doi: 10.1145/3620665.3640366. URL <https://doi.org/10.1145/3620665.3640366>.
- Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer. *CoRR*, abs/2004.05150, 2020. URL <https://arxiv.org/abs/2004.05150>.
- Buluç, A., Fineman, J. T., Frigo, M., Gilbert, J. R., and Leiserson, C. E. Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks. In auf der Heide, F. M. and Bender, M. A. (eds.), *SPAA 2009: Proceedings of the 21st Annual ACM Symposium on Parallelism in Algorithms and Architectures, Calgary, Alberta, Canada, August 11-13, 2009*, pp. 233–244.

- ACM, 2009. doi: 10.1145/1583991.1584053. URL <https://doi.org/10.1145/1583991.1584053>.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *CoRR*, abs/2401.10774, 2024. doi: 10.48550/ARXIV.2401.10774. URL <https://doi.org/10.48550/arXiv.2401.10774>.
- Chen, Z., Qu, Z., Liu, L., Ding, Y., and Xie, Y. Efficient tensor core-based GPU kernels for structured sparsity under reduced precision. In de Supinski, B. R., Hall, M. W., and Gamblin, T. (eds.), *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2021, St. Louis, Missouri, USA, November 14-19, 2021*, pp. 78. ACM, 2021. doi: 10.1145/3458817.3476182. URL <https://doi.org/10.1145/3458817.3476182>.
- Chen, Z., May, A., Svirschevski, R., Huang, Y., Ryabinin, M., Jia, Z., and Chen, B. Sequoia: Scalable, robust, and hardware-aware speculative decoding. *CoRR*, abs/2402.12374, 2024. doi: 10.48550/ARXIV.2402.12374. URL <https://doi.org/10.48550/arXiv.2402.12374>.
- Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: An open-source chatbot impressing gpt-4 with 90%\* chatgpt quality, March 2023. URL <https://lmsys.org/blog/2023-03-30-vicuna/>.
- Colfax. Cutlass tutorial: Mastering the nvidia tensor memory accelerator (tma), 2024. URL <https://research.colfax-intl.com/tutorial-hopper-tma/>.
- Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning. *CoRR*, abs/2307.08691, 2023. doi: 10.48550/ARXIV.2307.08691. URL <https://doi.org/10.48550/arXiv.2307.08691>.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL [http://papers.nips.cc/paper\\_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html](http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html).
- Dao, T., Haziza, D., Massa, F., and Sizov, G. Flash-decoding for long-context inference, 2023.
- DeepSeek-AI, Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Deng, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Yang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Chen, J., Yuan, J., Qiu, J., Song, J., Dong, K., Gao, K., Guan, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Pan, R., Xu, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Zheng, S., Wang, T., Pei, T., Yuan, T., Sun, T., Xiao, W. L., Zeng, W., An, W., Liu, W., Liang, W., Gao, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Chen, X., Nie, X., and Sun, X. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *CoRR*, abs/2405.04434, 2024. doi: 10.48550/ARXIV.2405.04434. URL <https://doi.org/10.48550/arXiv.2405.04434>.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Rozière, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Al-lonsius, D., Song, D., Pintz, D., Livshits, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Tsvvorn, H., Zarov, I., Ibarra, I. A., Kloumann, I. M., Misra, I., Evtimov, I., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., and et al. The llama 3 herd of models. *CoRR*, abs/2407.21783,



2024. doi: 10.48550/ARXIV.2407.21783. URL <https://doi.org/10.48550/arXiv.2407.21783>.
- Gray, A. Getting Started with CUDA Graphs | NVIDIA Technical Blog. <https://developer.nvidia.com/blog/cuda-graphs/>, 2019. [Accessed 19-10-2024].
- Gray, S., Radford, A., and Kingma, D. P. Gpu kernels for block-sparse weights. *arXiv preprint arXiv:1711.09224*, 3(2):2, 2017.
- Gupta, M. Mixed-input matrix multiplication performance optimizations. <https://research.google/blog/mixed-input-matrix-multiplication-performance-optimizations/>, January 2024. Google Research Blog, Accessed: 2024-01-26.
- He, H., Guessous, D., Liang, Y., and Dong, J. Flexattention: The flexibility of pytorch with the performance of flashattention, Aug 2024. URL <https://pytorch.org/blog/flexattention/>.
- Hong, K., Dai, G., Xu, J., Mao, Q., Li, X., Liu, J., chen, k., Dong, Y., and Wang, Y. Flashdecoding++: Faster large language model inference with asynchronization, flat gemm optimization, and heuristics. In Gibbons, P., Pekhimenko, G., and Sa, C. D. (eds.), *Proceedings of Machine Learning and Systems*, volume 6, pp. 148–161, 2024. URL [https://proceedings.mlsys.org/paper\\_files/paper/2024/file/5321b1dabc2be188d796c21b733e8c7-Paper-Conference.pdf](https://proceedings.mlsys.org/paper_files/paper/2024/file/5321b1dabc2be188d796c21b733e8c7-Paper-Conference.pdf).
- Im, E., Yelick, K. A., and Vuduc, R. W. Sparsity: Optimization framework for sparse matrix kernels. *Int. J. High Perform. Comput. Appl.*, 18(1):135–158, 2004. doi: 10.1177/1094342004041296. URL <https://doi.org/10.1177/1094342004041296>.
- Juravsky, J., Brown, B. C. A., Ehrlich, R. S., Fu, D. Y., Ré, C., and Mirhoseini, A. Hydragen: High-throughput LLM inference with shared prefixes. *CoRR*, abs/2402.05099, 2024. doi: 10.48550/ARXIV.2402.05099. URL <https://doi.org/10.48550/arXiv.2402.05099>.
- Kamath, A. K., Prabhu, R., Mohan, J., Peter, S., Ramjee, R., and Panwar, A. Pod-attention: Unlocking full prefill-decode overlap for faster llm inference. *CoRR*, abs/2410.18038, 2024. URL <https://arxiv.org/abs/2410.18038>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In Flinn, J., Seltzer, M. I., Druschel, P., Kaufmann, A., and Mace, J. (eds.), *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pp. 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Lai, R., Shao, J., Feng, S., Lyubomirsky, S. S., Hou, B., Lin, W., Ye, Z., Jin, H., Jin, Y., Liu, J., Jin, L., Cai, Y., Jiang, Z., Wu, Y., Park, S., Srivastava, P., Roesch, J. G., Mowry, T. C., and Chen, T. Relax: Composable abstractions for end-to-end dynamic machine learning. *CoRR*, abs/2311.02103, 2023. doi: 10.48550/ARXIV.2311.02103. URL <https://doi.org/10.48550/arXiv.2311.02103>.
- Li, S., Osawa, K., and Hoefer, T. Efficient quantized sparse matrix operations on tensor cores. In Wolf, F., Shende, S., Culhane, C., Alam, S. R., and Jagode, H. (eds.), *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis, Dallas, TX, USA, November 13-18, 2022*, pp. 37:1–37:15. IEEE, 2022. doi: 10.1109/SC41404.2022.00042. URL <https://doi.org/10.1109/SC41404.2022.00042>.
- Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., and Qiu, L. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 929–945, Santa Clara, CA, July 2024. USENIX Association. ISBN 978-1-939133-40-3. URL <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>.
- Liu, H. and Abbeel, P. Blockwise parallel transformers for large context models. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL [http://papers.nips.cc/paper\\_files/paper/2023/hash/1bfd87d2d92f0556819467dc08034f76-Abstract-Conference.html](http://papers.nips.cc/paper_files/paper/2023/hash/1bfd87d2d92f0556819467dc08034f76-Abstract-Conference.html).
- Liu, H., Zaharia, M., and Abbeel, P. Ring attention with blockwise transformers for near-infinite context. *CoRR*, abs/2310.01889, 2023. doi: 10.48550/ARXIV.2310.01889. URL <https://doi.org/10.48550/arXiv.2310.01889>.
- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., Zhu, A., Yang,

- L., Shi, X., Shi, C., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In Gupta, R., Abu-Ghazaleh, N. B., Musuvathi, M., and Tsafir, D. (eds.), *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, pp. 932–949. ACM, 2024. doi: 10.1145/3620666.3651335. URL <https://doi.org/10.1145/3620666.3651335>.
- Micikevicius, P., Stosic, D., Burgess, N., Cornea, M., Dubey, P., Grisenthwaite, R., Ha, S., Heinecke, A., Judd, P., Kamalu, J., Mellempudi, N., Oberman, S. F., Shoenybi, M., Siu, M. Y., and Wu, H. FP8 formats for deep learning. *CoRR*, abs/2209.05433, 2022. doi: 10.48550/ARXIV.2209.05433. URL <https://doi.org/10.48550/arXiv.2209.05433>.
- Milakov, M. and Gimelshein, N. Online normalizer calculation for softmax. *CoRR*, abs/1805.02867, 2018. URL <http://arxiv.org/abs/1805.02867>.
- Mirchandaney, R., Saltz, J. H., Smith, R. M., Nicol, D. M., and Crowley, K. Principles of runtime support for parallel processors. In Lenfant, J. (ed.), *Proceedings of the 2nd international conference on Supercomputing, ICS 1988, Saint Malo, France, July 4-8, 1988*, pp. 140–152. ACM, 1988. doi: 10.1145/55364.55378. URL <https://doi.org/10.1145/55364.55378>.
- MLC Community. Optimizing and characterizing high-throughput low-latency LLM inference in MLC Engine, Oct 2024. URL <https://blog.mlc.ai/2024/10/10/optimizing-and-characterizing-high-throughput-low-latency-llm-inference>. [Online; accessed February 19, 2026].
- Mostafa, H. Sequential aggregation and rematerialization: Distributed full-batch training of graph neural networks on large graphs. In Marculescu, D., Chi, Y., and Wu, C. (eds.), *Proceedings of Machine Learning and Systems 2022, MLSys 2022, Santa Clara, CA, USA, August 29 - September 1, 2022*. mlsys.org, 2022. URL [https://proceedings.mlsys.org/paper\\_files/paper/2022/hash/1d781258d409a6efc66cd1aa14a1681c-Abstract.html](https://proceedings.mlsys.org/paper_files/paper/2022/hash/1d781258d409a6efc66cd1aa14a1681c-Abstract.html).
- Narang, S., Undersander, E., and Diamos, G. F. Block-sparse recurrent neural networks. *CoRR*, abs/1711.02782, 2017. URL <http://arxiv.org/abs/1711.02782>.
- Nguyen, V., Carilli, M., Eryilmaz, S. B., Singh, V., Lin, M., Gimelshein, N., Desmaison, A., and Yang, E. Accelerating PyTorch with CUDA Graphs. <https://pytorch.org/blog/accelerating-pytorch-with-cuda-graphs/>, 2021. [Accessed 19-10-2024].
- NVIDIA. FasterTransformer. <https://github.com/NVIDIA/FasterTransformer>, 2021.
- NVIDIA. Nvidia hopper architecture in-depth, 2022. URL <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>.
- NVIDIA. NVIDIA TensorRT-LLM, 2023a. URL <https://docs.nvidia.com/tensorrt-llm/index.html>. [Online; accessed February 19, 2026].
- NVIDIA. Matrix multiplication background user’s guide, 2023b. URL <https://docs.nvidia.com/deeplearning/performance/pdf/Matrix-Multiplication-Background-User-Guide.pdf>.
- NVIDIA. Spatial partitioning (also known as warp specialization), 2024a. URL <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#spatial-partitioning-also-known-as-warp-specialization>.
- NVIDIA. New xqa-kernel provides 2.4x more llama-70b throughput within the same latency budget, 2024b. URL <https://github.com/NVIDIA/TensorRT-LLM/blob/main/docs/source/blogs/XQA-kernel.md>.
- Osama, M., Merrill, D., Cecka, C., Garland, M., and Owens, J. D. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the GPU. In Dehnavi, M. M., Kulkarni, M., and Krishnamoorthy, S. (eds.), *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPOPP 2023, Montreal, QC, Canada, 25 February 2023 - 1 March 2023*, pp. 429–431. ACM, 2023. doi: 10.1145/3572848.3577479. URL <https://doi.org/10.1145/3572848.3577479>.
- Ozen, G. Nvdsl: Simplifying tensor cores with python-driven mlir metaprogramming. In *Efficient Systems for Foundation Models (ES-FoMo) Workshop at ICML 2024*, 2024.
- Ponnusamy, R., Saltz, J. H., and Choudhary, A. N. Runtime compilation techniques for data partitioning and communication schedule reuse. In Borchers,

- B. and Crawford, D. (eds.), *Proceedings Supercomputing '93, Portland, Oregon, USA, November 15-19, 1993*, pp. 361–370. ACM, 1993. doi: 10.1145/169627.169752. URL <https://doi.org/10.1145/169627.169752>.
- Prabhu, R., Nayak, A., Mohan, J., Ramjee, R., and Panwar, A. vattention: Dynamic memory management for serving llms without pagedattention, 2024.
- Press, O., Smith, N. A., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=R8sQPpGCv0>.
- PyTorch-Labs. attention-gym. <https://github.com/pytorch-labs/attention-gym>, 2024.
- Rahman, M. K., Sujon, M. H., and Azad, A. Fusedmm: A unified sddmm-spm kernel for graph embedding and graph neural networks. In *35th IEEE International Parallel and Distributed Processing Symposium, IPDPS 2021, Portland, OR, USA, May 17-21, 2021*, pp. 256–266. IEEE, 2021. doi: 10.1109/IPDPS49936.2021.00034. URL <https://doi.org/10.1109/IPDPS49936.2021.00034>.
- Ramapuram, J., Danieli, F., Dhekane, E. G., Weers, F., Busbridge, D., Ablin, P., Likhomanenko, T., Digani, J., Gu, Z., Shidani, A., and Webb, R. Theory, analysis, and best practices for sigmoid self-attention. *CoRR*, abs/2409.04431, 2024. doi: 10.48550/ARXIV.2409.04431. URL <https://doi.org/10.48550/arXiv.2409.04431>.
- Rivière, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., Ferret, J., Liu, P., Tafti, P., Friesen, A., Casbon, M., Ramos, S., Kumar, R., Lan, C. L., Jerome, S., Tsitsulin, A., Vieillard, N., Stanczyk, P., Girgin, S., Momchev, N., Hoffman, M., Thakoor, S., Grill, J., Neyshabur, B., Bachem, O., Walton, A., Severyn, A., Parrish, A., Ahmad, A., Hutchinson, A., Abdagic, A., Carl, A., Shen, A., Brock, A., Coenen, A., Laforge, A., Paterson, A., Bastian, B., Piot, B., Wu, B., Royal, B., Chen, C., Kumar, C., Perry, C., Welty, C., Choquette-Choo, C. A., Sinopalnikov, D., Weinberger, D., Vijaykumar, D., Rogozinska, D., Herbison, D., Bandy, E., Wang, E., Noland, E., Moreira, E., Senter, E., Eltyshiev, E., Visin, F., Rasskin, G., Wei, G., Cameron, G., Martins, G., Hashemi, H., Klimczak-Plucinska, H., Batra, H., Dhand, H., Nardini, I., Mein, J., Zhou, J., Svensson, J., Stanway, J., Chan, J., Zhou, J. P., Carrasqueira, J., Iljazi, J., Becker, J., Fernandez, J., van Amersfoort, J., Gordon, J., Lipschultz, J., Newlan, J., Ji, J., Mohamed, K., Badola, K., Black, K., Millican, K., McDonell, K., Nguyen, K., Sodhia, K., Greene, K., Sjöstrand, L. L., Usui, L., Sifre, L., Heuermann, L., Lago, L., and McNealus, L. Gemma 2: Improving open language models at a practical size. *CoRR*, abs/2408.00118, 2024. doi: 10.48550/ARXIV.2408.00118. URL <https://doi.org/10.48550/arXiv.2408.00118>.
- Saltz, J. H. and Mirchandaney, R. The preprocessed doacross loop. In *Proceedings of the International Conference on Parallel Processing, ICPP '91, Austin, Texas, USA, August 1991. Volume II: Software*, pp. 174–179. CRC Press, 1991.
- Sanovar, R., Bharadwaj, S., Amant, R. S., Rühle, V., and Rajmohan, S. Lean attention: Hardware-aware scalable attention mechanism for the decode-phase of transformers. *CoRR*, abs/2405.10480, 2024. doi: 10.48550/ARXIV.2405.10480. URL <https://doi.org/10.48550/arXiv.2405.10480>.
- Setaluri, R., Aanjaneya, M., Bauer, S., and Sifakis, E. Spgrid: a sparse paged grid structure applied to adaptive smoke simulation. *ACM Trans. Graph.*, 33(6):205:1–205:12, 2014. doi: 10.1145/2661229.2661269. URL <https://doi.org/10.1145/2661229.2661269>.
- Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *CoRR*, abs/2407.08608, 2024. doi: 10.48550/ARXIV.2407.08608. URL <https://doi.org/10.48550/arXiv.2407.08608>.
- Shazeer, N. Fast transformer decoding: One write-head is all you need. *CoRR*, abs/1911.02150, 2019. URL <http://arxiv.org/abs/1911.02150>.
- Spector, B., Singhal, A., Arora, S., and Re, C. ThunderKittens: A Simple Embedded DSL for AI kernels, May 2024. URL <https://hazyresearch.stanford.edu/blog/2024-05-12-quick-tk>.
- Su, J., Ahmed, M. H. M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/J.NEUCOM.2023.127063. URL <https://doi.org/10.1016/j.neucom.2023.127063>.
- Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., and Han, S. QUEST: query-aware sparsity for efficient



- long-context LLM inference. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=KzACYwOMTV>.
- Tensorflow Developers. Ragged tensors | tensorflow core. [https://www.tensorflow.org/guide/ragged\\_tensor](https://www.tensorflow.org/guide/ragged_tensor), 2018.
- Thakkar, V., Ramani, P., Cecka, C., Shivam, A., Lu, H., Yan, E., Kosaian, J., Hoemmen, M., Wu, H., Kerr, A., Nicely, M., Merrill, D., Blasig, D., Qiao, F., Majcher, P., Springer, P., Hohnerbach, M., Wang, J., and Gupta, M. CUTLASS, January 2023. URL <https://github.com/NVIDIA/cutlass>.
- Tillet, P., Kung, H. T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, pp. 10–19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367196. doi: 10.1145/3315508.3329973. URL <https://doi.org/10.1145/3315508.3329973>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In Guyon, I., von Luxburg, U., Bengio, S., Wallach, H. M., Fergus, R., Vishwanathan, S. V. N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 5998–6008, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- Wang, Y., Feng, B., Wang, Z., Huang, G., and Ding, Y. TC-GNN: bridging sparse GNN computation and dense tensor cores on gpus. In Lawall, J. and Williams, D. (eds.), *2023 USENIX Annual Technical Conference, USENIX ATC 2023, Boston, MA, USA, July 10-12, 2023*, pp. 149–164. USENIX Association, 2023. URL <https://www.usenix.org/conference/atc23/presentation/wang-yuke>.
- Wu, M., Cheng, X., Padon, O., and Jia, Z. A multi-level superoptimizer for tensor programs. *CoRR*, abs/2405.05751, 2024. doi: 10.48550/ARXIV.2405.05751. URL <https://doi.org/10.48550/arXiv.2405.05751>.
- xAI. Open Release of Grok-1. <https://x.ai/blog/grok-os>, 2023. [Accessed 24-06-2024].
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv*, 2023.
- Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. Gated linear attention transformers with hardware-efficient training, 2024. URL <https://arxiv.org/abs/2312.06635>.
- Ye, L., Tao, Z., Huang, Y., and Li, Y. Chunkattention: Efficient self-attention with prefix-aware KV cache and two-phase partition. In Ku, L., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pp. 11608–11620. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.623. URL <https://doi.org/10.18653/v1/2024.acl-long.623>.
- Ye, Z., Lai, R., Shao, J., Chen, T., and Ceze, L. Sparsatir: Composable abstractions for sparse compilation in deep learning. In Aamodt, T. M., Jerger, N. D. E., and Swift, M. M. (eds.), *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, pp. 660–678. ACM, 2023. doi: 10.1145/3582016.3582047. URL <https://doi.org/10.1145/3582016.3582047>.
- Yu, G., Jeong, J. S., Kim, G., Kim, S., and Chun, B. Orca: A distributed serving system for transformer-based generative models. In Aguilera, M. K. and Weatherspoon, H. (eds.), *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, pp. 521–538. USENIX Association, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- Zhai, Y., Jiang, C., Wang, L., Jia, X., Zhang, S., Chen, Z., Liu, X., and Zhu, Y. Bytetransformer: A high-performance transformer boosted for variable-length inputs. In *IEEE International Parallel and Distributed Processing Symposium, IPDPS 2023, St. Petersburg, FL, USA, May 15-19, 2023*, pp. 344–355. IEEE, 2023. doi: 10.1109/IPDPS54959.2023.00042. URL <https://doi.org/10.1109/IPDPS54959.2023.00042>.
- Zhang, H., Yu, Z., Dai, G., Huang, G., Ding, Y., Xie, Y., and Wang, Y. Understanding GNN computational graph: A coordinated computation, io, and memory perspective. In Marculescu,

D., Chi, Y., and Wu, C. (eds.), *Proceedings of Machine Learning and Systems 2022, ML-Sys 2022, Santa Clara, CA, USA, August 29 - September 1, 2022*. mlsys.org, 2022. URL [https://proceedings.mlsys.org/paper\\_files/paper/2022/hash/b559156047e50cf316207249d0b5a6c5-Abstract.html](https://proceedings.mlsys.org/paper_files/paper/2022/hash/b559156047e50cf316207249d0b5a6c5-Abstract.html).

Zheng, L., Chiang, W., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging llm-as-a-judge with mt-bench and chatbot arena. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023a. URL [http://papers.nips.cc/paper\\_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets\\_and\\_Benchmarks.html](http://papers.nips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html).

Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C. W., and Sheng, Y. Efficiently programming large language models using sglang. *CoRR*, abs/2312.07104, 2023b. doi: 10.48550/ARXIV.2312.07104. URL <https://doi.org/10.48550/arXiv.2312.07104>.

Zhu, K., Zhao, Y., Zhao, L., Zuo, G., Gu, Y., Xie, D., Gao, Y., Xu, Q., Tang, T., Ye, Z., Kamahori, K., Lin, C., Wang, S., Krishnamurthy, A., and Kasikci, B. Nanoflow: Towards optimal large language model serving throughput. *CoRR*, abs/2408.12757, 2024a. doi: 10.48550/ARXIV.2408.12757. URL <https://doi.org/10.48550/arXiv.2408.12757>.

Zhu, L., Wang, X., Zhang, W., and Lau, R. W. H. Relayattention for efficient large language model serving with long system prompts. In Ku, L., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pp. 4945–4957. Association for Computational Linguistics, 2024b. doi: 10.18653/V1/2024.ACL-LONG.270. URL <https://doi.org/10.18653/v1/2024.acl-long.270>.

## A 分组查询注意力中的头组融合

分组查询注意力 (GQA) (Ainslie et al., 2023) 允许多个 query 头共享同一组 key-value (KV) 头。一个直接实现是为每个 query 头分配独立 GPU threadblock; 但当 query 长度较短时, 这会使大量潜在 KV-Cache 复用无法被利用。为解决该问题, FlashInfer 提供头组融合 (*head-group fusion*) 策略: 将不同 KV 头映射到独立 threadblock, 同时把 query 头维与 query 长度维进行融合。图 11 展示了该融合方案中融合后行索引与原始行索引、头索引之间的关系。通过在线程块映射中合并 query-head 维与行维, 组内所有 query 头可共享一次 KV-Cache 的共享内存加载, 从而提升内存复用与 GQA 吞吐。

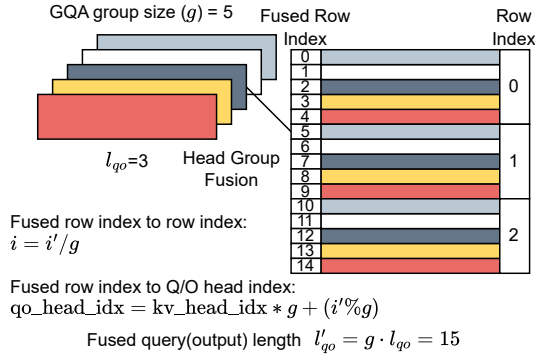


Figure 11. FlashInfer 在 GQA 中将 query 头与 query 长度维进行头组融合。

我们主要在短 query 长度场景下使用头组融合。当 query 长度足够大时, query 维本身已能提供足够工作量来有效利用 KV-Cache, 头组融合的重要性会降低。类似思想也在其他框架中出现, 例如 TensorRT-LLM (NVIDIA, 2023a) 中的 XQA (NVIDIA, 2024b)。

## B 稀疏 Gather 的开销

在第 3.2.1 节中, 我们详细介绍了 FlashInfer 的稀疏加载模块: 将全局内存中的稀疏行搬运到连续共享内存。本节测量 FlashInfer 在 *decode* 与 *prefill* 两类内核下由稀疏 gather 引入的性能开销。

图 12 对比了稀疏与稠密 (连续) KV-Cache 在 prefill 与 decode 内核下的吞吐。prefill 内核使用 *causal* 注意力场景, 这是 LLM 服务中的常见设置。对于连续 KV-Cache, 我们使用变长 RaggedTensor prefill API<sup>8</sup>; 对于稀疏 KV-Cache, 我们使用 PagedKVCache prefill

<sup>8</sup><https://docs.flashinfer.ai/api/prefill.html#flashinfer.prefill.BatchPrefillWithRaggedKVCacheWrapper>

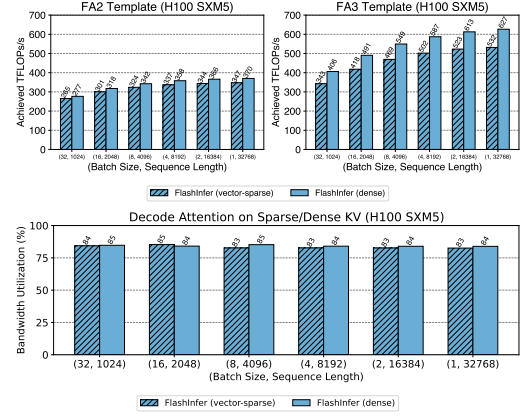


Figure 12. 上: 在 FA2/FA3 模板下, 使用稠密/稀疏 KV-Cache 的 (因果) prefill 注意力内核达到的 TFLOPs/s。下: 使用稠密/稀疏 KV-Cache 的 decode 注意力内核带宽利用率。稀疏 KV-Cache 采用 page size 为 1 (向量稀疏) 的 PageAttention。横轴为不同 batch size 与序列长度。

## API<sup>9</sup>

query 头数与 KV 头数固定为 32, head dimension 设为 128。我们变化 batch size 与序列长度测量吞吐。decode 内核中, 稀疏与稠密 KV-Cache 的性能差距可以忽略 (在 1% 以内); prefill 内核中, 约有 10% 的差距。

需要注意, FA3 模板中的稠密注意力在 key/value 加载时使用 TMA (Tensor Memory Access) 指令, 而稀疏 gather 无法使用该机制, 因为 Hopper 的 TMA 仅支持定步长访问, 而稀疏 gather 需要任意行索引。因此, FA3 稀疏 gather 需回退到 Ampere 风格异步拷贝与手动指针运算, 这会增加寄存器消耗, 并需减小 KV tile 大小以避免寄存器溢出, 导致性能差距略大。相比之下, 在 FA2 模板中 (稀疏与稠密都使用 Ampere 异步拷贝) 差距更小, 因为二者使用相同 tile 大小。

当块稀疏矩阵的列块大小较大 (例如 128 及以上) 时, 稀疏 gather 可使用 TMA, 因为每条 TMA 指令都可在单个固定步长块内工作。我们将该优化留作未来工作。不过, 提高列块大小会降低块稀疏格式灵活性, 不一定适用于所有场景。

## C 后端选择

在 NVIDIA GPU 上, 我们选择在 CUDA/CUTLASS (Thakkar et al., 2023) 之上构建 FlashInfer, 而非 Triton (Tillet et al., 2019), 主要原因如下:

1. 更完整的 NVIDIA 新特性支持。CUTLASS 支持 warp-specialization (NVIDIA, 2024a),

<sup>9</sup><https://docs.flashinfer.ai/api/prefill.html#flashinfer.prefill.BatchPrefillWithPagedKVCacheWrapper>



TMA (NVIDIA, 2022) 等专用 GPU 能力, 而这些能力在 Triton 中当前仍处于实验阶段或尚未支持。

2. **更细粒度的内核优化能力。** Triton 提供 tile 级抽象, 而 CUDA/CUTLASS 可进一步细化到线程级寄存器控制。这样的灵活性使我们更容易将低层优化 (如 PTX intrinsic) 直接并入 JIT 模板, 这在 Triton 中更困难。

我们的负载均衡调度器设计 (第 3.3.1 节) 在很大程度上与后端无关, 因此未来 FlashInfer 可以集成 Triton, 也可迁移到其他硬件平台。

## D 内存管理

FlashInfer 管理一块页锁定 (pinned) 主机缓冲区和一块设备 workspace 缓冲区, 用于存储调度器元数据与 split-k 部分输出。我们将设备 workspace 划分为多个 section, 每个 section 对应一组调度元数据数组或 split-k 部分输出数组。每次调度器 plan 调用时, 我们先在 pinned host buffer 上生成调度元数据, 再通过 cudaMemcpyAsync 异步拷贝到设备 workspace 的对应 section。

### D.1 兼容 CUDAGraph 的 Workspace 布局

一旦内核被 CUDA Graph 捕获, 其参数 (指针与标量) 会固定, 因此设备 workspace 各 section 的地址在整个图生命周期内必须保持稳定。为此, 我们按上界估计为每个 section 分配其最大容量 (覆盖调度元数据与部分输出)。

### D.2 Split-K 的直写优化

在 FlashInfer 的负载均衡调度器中 (第 3.3.1 节), 仅 KV 长度较大的请求会进行 KV 切分。KV 较短请求无需切分, 因此没有部分输出归约步骤。为节省计算与 workspace 内存, 这些短请求可将部分输出直接写入最终输出缓冲区 (绕过设备 workspace)。该策略同时降低了 workspace 需求与 contraction 内核计算负载。

### D.3 Workspace 缓冲区大小估计

workspace 大小主要由两部分决定: (1) 调度元数据所需空间; (2) split-k 部分输出所需空间。

**调度元数据。** 每个元数据 section 的最大大小由“最大并发请求数”与“最大累计请求长度”决定。用户需在调度器首次计划阶段提供这些上界。

**部分输出。** 部分输出大小取决于问题维度 (头数与 head dimension) 以及每次内核启动的 CTA 数。按我们的负载均衡算法 3.3.1, 只有“长请求” (即 KV 长度超过总 KV 长度除以 CTA 数) 才会被切分。根据第 D.2

节的直写优化, 只有这些切分请求会在 workspace 中产生输出。由于切分数不超过 CTA 总数, 且每次切分最多产生两个需合并的 tile, 因此部分输出最多为  $2 \times \#CTA$ 。每个 tile 产生大小为  $T_q \cdot H_{qo} \cdot (D+1)$  的部分输出, 其中  $T_q$  是 query tile 大小,  $H_{qo}$  为头数,  $D+1$  为 head dimension 加 LSE 维度。因此部分输出总大小上界为:

$$2 \#CTA \times T_q \times H_{qo} \times (D+1).$$

默认情况下, CTA 总数设为  $k \times \#SM$ , 其中  $\#SM$  为 GPU 上流式多处理器数量,  $k$  选择为可最大化 CTA 级占用率。对于寄存器占用高的 Tensor Core 微内核, Ampere 上  $k$  通常不超过 2; Hopper 上通常为 1 (每个 SM 一个 CTA, 也称持久化内核)。

## E 注意力与其他算子的重叠执行

Nanoflow (Zhu et al., 2024a) 在不同 CUDA stream 中重叠执行 GEMM、注意力与跨设备通信, 并为每类操作分配固定 SM 数量。在 FlashInfer 中, 用户可通过 plan 函数传入该 SM 数, FlashInfer 的负载均衡调度器会据此分配 tile。

## F FP8-FP16 混合精度注意力

近期 LLM 常采用 fp8 KV-Cache 来降低内存带宽与存储成本 (Micikevicius et al., 2022)。在 FlashInfer 中, 我们实现了混合精度注意力内核: query 与 output 保持 fp16, KV-Cache 存储为 fp8。我们利用 Gupta (2024) 提出的快速数值数组转换器与 fragment shuffler 来加速反量化并高效处理位宽不匹配。该设计可在不显著损失数值精度的情况下, 降低内存占用并提升带宽利用。

## G 额外评估

本节给出更多实验结果, 以进一步验证 FlashInfer 在不同实验条件下的性能、可扩展性与鲁棒性。

### G.1 与 FlexAttention 的比较

我们在 NVIDIA H100 80GB SXM 上使用 AttentionGym (PyTorch-Labs, 2024) 基准比较 FlashInfer 与 FlexAttention (He et al., 2024) 在不同注意力变体下的性能。评测配置为 batch size 16、头数 16、head dim 128, CUDA 版本与 Triton 版本分别固定为 12.4 与 3.2。表 1 至 4 给出 FlashInfer 与 FlexAttention 的 TFLOPS/s (越高越好)。在四类场景和多种序列长度下, FlashInfer 均稳定优于 FlexAttention, 且在长序列下优势更明显。性能提升主要来自对 Hopper 架构高级特性的利用 (如 warp specialization 与 TMA), 以及 CUTLASS 相比 Triton 更细粒度的资源控制 (寄存器级而非 tile 级)。随着 Triton 对这些特性的完整支持, 差距预计会缩小。

Table 1. 因果注意力

| 序列长度  | FlexAttention | FlashInfer     |
|-------|---------------|----------------|
| 512   | 209.11        | <b>250.454</b> |
| 1024  | 294.53        | <b>406.554</b> |
| 2048  | 376.90        | <b>487.236</b> |
| 4096  | 421.00        | <b>548.388</b> |
| 8192  | 441.26        | <b>587.903</b> |
| 16384 | 453.57        | <b>612.259</b> |

Table 2. 带 Logits SoftCap 的注意力

| 序列长度  | FlexAttention | FlashInfer     |
|-------|---------------|----------------|
| 512   | 241.51        | <b>336.487</b> |
| 1024  | 327.50        | <b>409.534</b> |
| 2048  | 379.57        | <b>468.769</b> |
| 4096  | 403.39        | <b>489.667</b> |
| 8192  | 407.82        | <b>515.573</b> |
| 16384 | 409.89        | <b>520.935</b> |

## G.2 共享前缀注意力内核评估

我们在后缀长度为 128 下评估共享前缀注意力内核。表 5 给出了不同共享前缀长度、不同场景和不同 batch size 下的内核时延 (单位: 微秒, us), 其中“composable”表示可组合格式, “single”表示单格式。可组合格式在长前缀 (如 32k) 和大 batch (如 64) 下收益明显。不过这些内核级提升不总能线性转化为端到端收益, 因为真实场景中的共享前缀通常更短。

## G.3 变长序列与负载均衡调度消融

我们对负载均衡调度器 (第 3.3.1 节) 的效果进行消融。表 6 与 7 展示了 Llama 3.1-8B-Instruct 在 NVIDIA H100 SXM5 上、SGLang (Zheng et al., 2023b) + FlashInfer (启用/禁用负载均衡调度) 下的结果。我们评估三类数据集上的 ITL (ms) 与 TTFT (ms): ShareGPT、输入长度采样自  $U(512, 2048)$  且输出固定为 256 的变长集、输入长度采样自  $U(4096, 16384)$  且输出固定为 256 的变长集。表中“RR”表示请求率。

## G.4 vLLM 集成评估

我们在固定请求率 16 下比较 vLLM 的 FlashInfer 后端与默认后端, 并在表 8 报告吞吐 (tokens/s)、ITL (ms) 与 TTFT (ms)。在 fp8 KV-cache 下, FlashInfer 的 ITL 可降低约 13%; 但在 bf16 下, vLLM 集成中的主机侧 Python 开销 (如数组操作) 会带来轻微回退。未来我们将通过 C++ 化并将调度器下沉到设备端来优化。

Table 3. ALiBi 偏置 (Press et al., 2022)

| 序列长度  | FlexAttention | FlashInfer     |
|-------|---------------|----------------|
| 512   | 253.22        | <b>403.899</b> |
| 1024  | 344.70        | <b>500.220</b> |
| 2048  | 406.14        | <b>535.498</b> |
| 4096  | 426.13        | <b>561.324</b> |
| 8192  | 436.35        | <b>573.493</b> |
| 16384 | 434.86        | <b>578.005</b> |

Table 4. 滑动窗口 (窗口大小 = 1024)

| 序列长度  | FlexAttention | FlashInfer     |
|-------|---------------|----------------|
| 512   | 206.51        | <b>236.363</b> |
| 1024  | 292.25        | <b>374.108</b> |
| 2048  | 350.91        | <b>381.464</b> |
| 4096  | 368.45        | <b>384.998</b> |
| 8192  | 373.25        | <b>384.514</b> |
| 16384 | 367.91        | <b>380.506</b> |

## G.5 细粒度块稀疏评估

FlashInfer 支持细粒度块稀疏矩阵, 这对多种 KV-Cache 剪枝算法很有价值。我们在 Quest (Tang et al., 2024) 上测量内核性能。Quest 是一种 SOTA 长上下文建模算法, KV-Cache 采用细粒度稀疏。我们在 NVIDIA H100 SXM5 上对 Quest 的批量解码注意力内核进行比较, 评估 FlashInfer、PyTorch SDPA 与 FlexAttention, 配置为 (block size 16, num\_qo\_heads 32, num\_kv\_heads 32, head\_dim 128)。所有时延单位均为微秒 (us)。

如表 9 至 11 所示, FlashInfer 具有显著性能优势, 在长序列下最高可达 20x 加速。目前 FlexAttention 依赖大块大小模板, 而 FlashInfer 采用 sparse-row gather 策略, 在小块大小下也能利用稠密 Tensor Core。该设计天然支持细粒度 KV-cache 剪枝。

Table 5. 共享前缀注意力内核时延

| 共享前缀长度 | Composable (BS=16) | Single (BS=16) | Composable (BS=64) | Single (BS=64) |
|--------|--------------------|----------------|--------------------|----------------|
| 1024   | <b>45.17</b>       | 46.52          | 87.86              | 130.49         |
| 8192   | <b>88.67</b>       | 226.57         | 125.76             | 931.75         |
| 32768  | <b>217.42</b>      | 945.67         | 254.54             | 4090           |

Table 6. 负载均衡调度消融 (ITL)

| 场景                      | 启用<br>负载均衡  | 禁用<br>负载均衡 | Triton |
|-------------------------|-------------|------------|--------|
| ShareGPT (RR=16)        | <b>8.96</b> | 9.16       | 9.36   |
| $U(512, 2048)$ (RR=8)   | <b>8.21</b> | 8.42       | 8.49   |
| $U(4096, 16384)$ (RR=1) | <b>8.63</b> | 13.89      | 11.08  |

Table 7. 负载均衡调度消融 (TTFT)

| 场景                      | 启用<br>负载均衡    | 禁用<br>负载均衡 | Triton |
|-------------------------|---------------|------------|--------|
| ShareGPT (RR=16)        | <b>39.05</b>  | 39.42      | 52.92  |
| $U(512, 2048)$ (RR=8)   | <b>66.78</b>  | 67.38      | 68.48  |
| $U(4096, 16384)$ (RR=1) | <b>411.02</b> | 421.60     | 566.30 |

Table 8. vLLM 集成评估

| 后端                | 吞吐      | 中位<br>ITL | 中位<br>TTFT |
|-------------------|---------|-----------|------------|
| Default (bf16)    | 6062.89 | 10.42     | 35.85      |
| FlashInfer (bf16) | 6065.41 | 10.63     | 36.60      |
| Default (e4m3)    | 6015.86 | 12.56     | 39.74      |
| FlashInfer (e4m3) | 6020.32 | 10.92     | 37.93      |

Table 9. FlashInfer 细粒度稀疏时延 (us)

| seq_len | page_budget |        |        |        |
|---------|-------------|--------|--------|--------|
|         | 64          | 128    | 256    | 512    |
| 4096    | 20.299      | 30.361 | 44.383 | 44.430 |
| 8192    | 22.273      | 28.603 | 44.928 | 68.194 |
| 16384   | 20.485      | 28.678 | 44.677 | 68.700 |
| 32768   | 22.371      | 28.700 | 44.988 | 68.478 |

Table 10. PyTorch SDPA 细粒度稀疏时延 (us)

| seq_len | page_budget |          |          |          |
|---------|-------------|----------|----------|----------|
|         | 64          | 128      | 256      | 512      |
| 4096    | 287.684     | 288.904  | 287.715  | 287.807  |
| 8192    | 474.631     | 474.508  | 474.683  | 473.070  |
| 16384   | 857.319     | 857.570  | 857.094  | 857.728  |
| 32768   | 1711.955    | 1711.621 | 1713.093 | 1711.709 |

Table 11. FlexAttention 细粒度稀疏时延 (us)

| seq_len | page_budget |          |          |          |
|---------|-------------|----------|----------|----------|
|         | 64          | 128      | 256      | 512      |
| 4096    | 1100.349    | 1097.356 | 1073.753 | 1071.797 |
| 8192    | 1092.695    | 1099.100 | 1078.081 | 1074.886 |
| 16384   | 1109.817    | 1101.535 | 1077.639 | 1076.859 |
| 32768   | 1169.109    | 1187.395 | 1176.332 | 1174.502 |